efrei
PARIS PANTHÉON-ASSAS UNIVERSITÉ

Java1: Fondamentaux de la POO

- Encapsulation : private, public
- Getters / Setters
- La méthode toString()
- mot clé @Override (la syntaxe de la signature)
- Comparaison d'objets
- Les attributs et méthodes statiques

Coordination : Cherifa BEN KHELIL
cherifa.ben-khelil@efrei.fr

Support préparé par : Asma Gabis asma.gabis@efrei.fr

Une partie de cette présentation est tirée du cours Java de l'EPFL

Programmation Orientée Objet

Introduction

- L'objectif principal de la notion d'objet est d'offrir des outils qui permettent d'organiser des programmes de façon plus efficace.

- Les objets permettent de mettre en œuvre dans les programmes les notions :
  - ✓ d'**encapsulation**
  - ✓ d'**abstraction**
  - ✓ d'**héritage**
  - ✓ de **polymorphisme**

01

L’encapsulation

01

L'encapsulation

Définition

- Le principe **d'encapsulation** consiste à regrouper dans le même objet informatique les données et traitements qui lui sont spécifiques :
  - ✓ Les données inclues dans un objet seront appelées les attributs de cet objet ;
  - ✓ Les traitements/fonctions défini(e)s dans un objet seront appelées les méthodes de cet objet.

Objet
=
Attributs
+
Méthodes

L'encapsulation

Notion d'abstraction

- Pour être véritablement intéressant, un **objet** doit permettre un certain degré *d'abstraction*.

- Le processus d'abstraction consiste à identifier pour un ensemble d'éléments :
  - ✓ des caractéristiques communes à tous les éléments
  - ✓ des mécanismes communs à tous les éléments

➔ Description générique de l'ensemble considéré : se focaliser sur l'essentiel, cacher les détails.

L'encapsulation

Notion d'abstraction

Exemple : **Rectangles**

- La notion d'« objet rectangle » n'est intéressante que si l'on peut lui associer des propriétés et/ou mécanismes généraux

  ➔ Propriétés et mécanismes **valables pour l'ensemble** des rectangles et non pas pour un rectangle particulier.

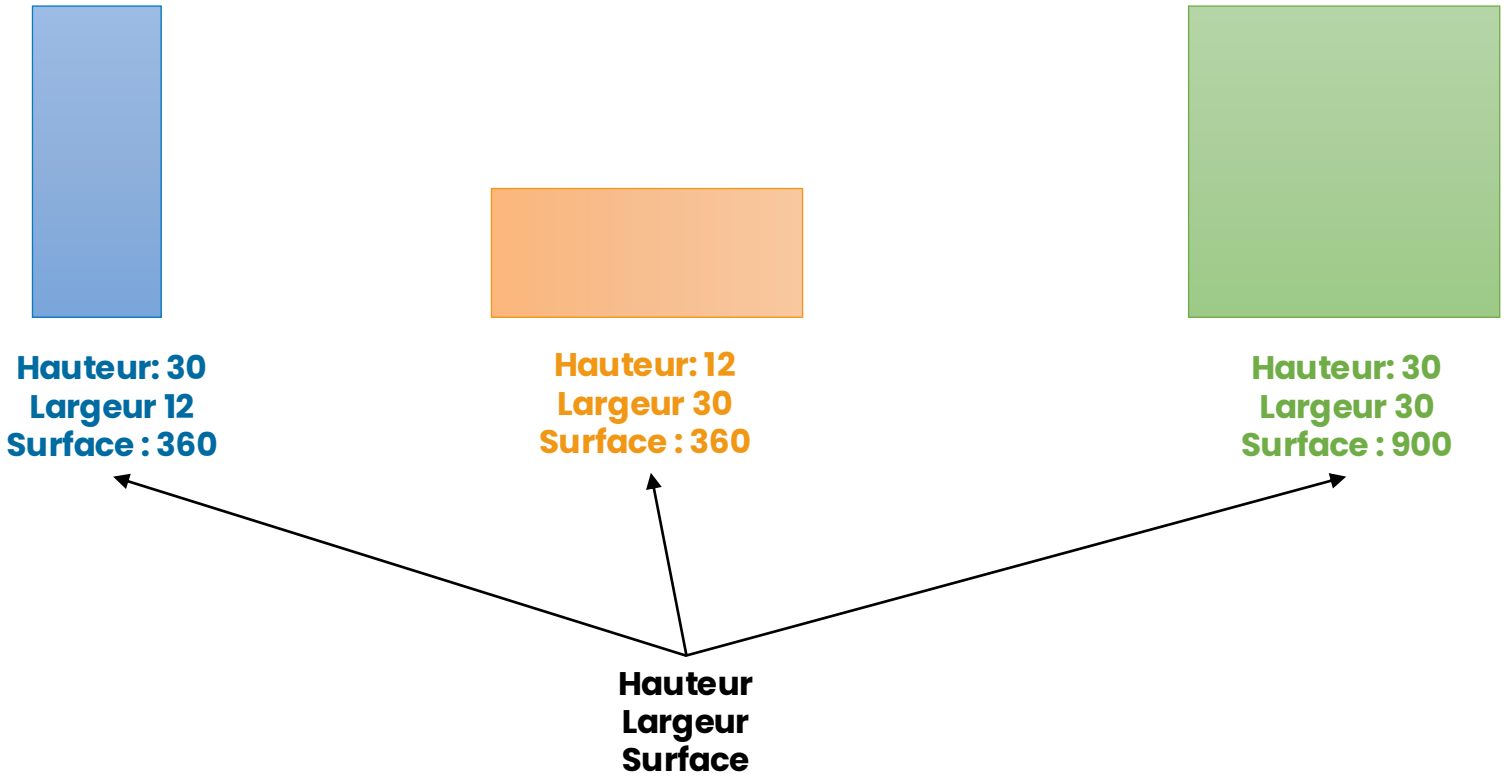
- Les notions de largeur et hauteur sont des propriétés générales des rectangles (**attributs**),

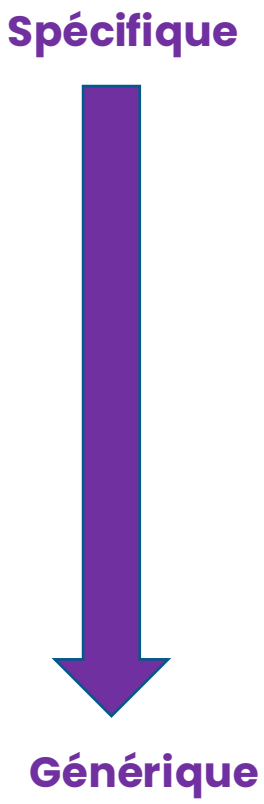
- Le mécanisme permettant de calculer la surface d'un rectangle (surface = largeur × hauteur) est commun à tous les rectangles (**méthodes**)

L'encapsulation

Notion d'abstraction

Illustration : Considérons les 3 rectangles suivants





L'encapsulation

Abstraction et encapsulation

- Un intérêt de **l'encapsulation** est que cela permet d'**abstraire** :
- En plus du regroupement des données et des traitements relatifs à une entité, l'encapsulation permet en effet de définir deux niveaux de perception :
  - ✓ **Un niveau externe** : partie « visible » (par les programmeurs-utilisateurs) de l'objet : le prototype des méthodes et attributs accessibles hors de l'objet
  - ✓ **Un niveau interne** : (détails d') implémentation de l'objet, méthodes et attributs accessibles uniquement depuis l'intérieur de l'objet (ou l'intérieur d'objets similaires)

    ➔ Corps de l'objet

L'encapsulation

Abstraction et encapsulation

Encapsulation

- Regroupement de tout ce qui caractérise l'objet: données (attributs) et traitements (méthodes)
- Isolement et dissimulation des détails d'implémentation

L'encapsulation

Comment encapsuler ?

- En cachant (ou non) l'existence de certains éléments de l'objet à d'autres objets.

➔ Jouer sur la **visibilité des membres de la classe**

4 niveaux de visibilité :

- **Public** : Aucune restriction d'accès. Accessible à partir d'un objet ou de n'importe quelle classe fille
- **Privé** : Ne peut être accédé qu'à partir de la classe qui le définit
- **Protected :** Accessible uniquement à partir de la classe qui la définit, ainsi que de toute classe enfant
- **Par défaut** : Accessible uniquement à partir du package actuel

L'encapsulation

Comment décider de la visibilité ?

Dans la plus part des cas , mettre :

- **Privé**
  - ✓ Tous les attributs de la classe systématiquement
  - ✓ Les méthodes de la classe ***non utilisées par l'application utilisateur***

- **Public**
  - ✓ Les méthodes de la classe ***utilisées par l'application utilisateur***

- **Protected**
  - ✓ Pour les attributs et méthodes d'une classe mère ***dans le cas d'un héritage*** qui ne sont pas utilisés par l'utilisateur mais uniquement par les classes filles.

L'encapsulation

L'accès public

Mettre le mot-clé « **public** » avant l'attribut et/ou la méthode concerné :

public typeAtt nomAtt ;

public typeSortie nomMethode(liste paramètres)

```
public class Point
{
    // Liste des attributs
    public double x;
    public double y;
    // Constructeur par défaut

    public Point()
    {}

    // Constructeur d'initialisation à deux paramètres
    public Point(int x, int y)
    {
        this.x = x;
        this.y = y;
    }
}
```

L'encapsulation

L'accès privé

Mettre le mot-clé « **private** » avant l'attribut et/ou la méthode concerné:

private typeAtt nomAtt ;

private typeSortie nomMethode(liste paramètres)

```
public class Point
{
    // Liste des attributs
    private double x;
    private double y;
}
```

```
public class Main
{
    public static void main(String[] args)
    {
        Point p = new Point();
    }
}
```

L'encapsulation

L'accès privé

```java
public class Point
{
    // Liste des attributs
    private double x;
    private double y;
}
```

```java
public class Main
{
    public static void main(String[] args)
    {
        Point p = new Point();
        p.x = 3;

        System.out.println(p.y);
    }
}
```

'y' has private access in 'com.company.Point'

Create field 'y' in 'Point' ⌥⇧⏎ More actions... ⌥⏎

com.company.Point
private double y

Comment faire pour modifier / afficher le contenu d'un attribut ?

02

Getters/ Setters «Accesseurs» /«Mutateurs »

02

Accesseurs / Mutateurs

Pour la lecture d'un attribut : accesseur/ getter

- Accesseur (« méthode get() » ou « getters »)
  - Consultation d'une variable (accès en lecture)
  - Retourne la valeur de l'attribut auquel il est associé

Syntaxe :

```
public typeDeRetour getNomAttrib()
{
        return nomAttrib;
}
```

Exemple :

```
public double getHauteur()
{
         return hauteur;
}
```

Accesseurs / Mutateurs

Pour la lecture d'un attribut : accesseur/ getter

Exemple :

```
class Rectangle
{
    private double largeur;
    private double hauteur;
    // constructeur
    Rectangle(double h, double l)
    {hauteur = h;
        largeur = l; }
    // getters
    public double getHauteur() {return hauteur;}
    public double getLargeur(){ return largeur;}

    // méthode surface
    public double surface ()  {
        return hauteur* largeur;
    }
}
```

```
class RectangleMain
{
    public static void main(String[] args)
    {
        Rectangle rect1 = new Rectangle(3.0,4.0);
        System.out.println(rect1.getHauteur());

        //...
    }
}
```

Accesseurs / Mutateurs

Pour l'écriture d'un attribut : mutateur / setter

- Modificateur « méthodes set() » ou « setters »
  - ✓ Modification
  - ✓ Affectation de l'argument à une variable d'instance précise

Syntaxe :

```
public void setNomAttrib(typeParam paramètre ){
        nomAttrib = paramètre ;
}
```

Exemples :

```
public void setLongueur(double lo) {
        longueur = lo;
}
```

```
public void setLongueur(double longueur) {
        this.longueur = longueur;
}
```

Accesseurs / Mutateurs

Pour l'écriture d'un attribut : mutateur / setter

Exemple :

```
class Rectangle {
    private double largeur;
    private double hauteur;
    // constructeur
    Rectangle(double h, double l)
    {hauteur = h;
        largeur = l; }
    // getters
    public double getHauteur() {return hauteur;}
    public double getLargeur(){ return largeur;}

    //setters
    public void setHauteur(double h)
    {   hauteur = h;}

    public void setLargeur(double la)
    {   largeur = la;}

    // méthode surface
    public double surface ()  {
        return hauteur* largeur;
    }
}
```

```
class RectangleMain
{
    public static void main(String[] args)
    {
        Rectangle rect = new Rectangle(3.0,4.0);

        rect.setHauteur(8.0);
        rect.setLargeur(7.0);

        System.out.println(rect.getHauteur());

    }
}
```

```
>>>8.0
```

03

Affichage d’un objet

03

Affichage d'un objet

Besoin

Soit la portion du code suivant:

```
Rectangle r = new Rectangle(3,4);
System.out.println(r);
```

```
>>>>> Rectangle@23fc625e
```

Que fait-on si on souhaite faire afficher le contenu d'un objet ?

Affichage d'un objet

Méthode toString()

- Java prévoit une méthode qui retourne une représentation de l'instance sous forme de String
- La méthode **toString()** est définie dans la classe Object ; en conséquence toutes les classes Java en hérite

```
@Override
public String toString()
{
    // Corps de la méthode
    return …
}
```

- Par défaut, la méthode **toString()** affiche la référence de l'objet
- Pour la personnaliser et lui permettre de retourner une chaîne de caractères décrivant les informations d'un objet, on procède à sa redéfinition.
- L'annotation @Override doit être mise au-dessus de toute méthode redéfinie (plus de détails plus tard ☺)

Affichage d'un objet

Méthode toString()

```
class Rectangle {
    private double largeur;
    private double hauteur;
    // constructeur
    Rectangle(double h, double l)
    {  hauteur = h;
        largeur = l;
    }

    @Override
    public String toString(){
        return "Rectangle " + hauteur + " x " +
largeur;
   }
}
```

```
class RectangleMain
{
    public static void main(String[] args) {

        Rectangle rect = new Rectangle(4.5,9.0);

        System.out.println(rect);

    }
}
```

```
>>> Rectangle 4.5 x 9.0
```

Affichage d'un objet

@Override

- L'annotation *@Override* indique au compilateur qu'une méthode donnée est la redéfinition d'une autre méthode de la superclasse.

- En plus de rappeler au programmeur qu'il s'agit de la redéfinition d'une autre méthode, la présence de l'annotation *@Override* va forcer le compilateur à vérifier que la méthode a été redéfinie correctement.

Affichage d'un objet

@Override

- L'annotation *@Override* indique au compilateur qu'une méthode donnée est la redéfinition d'une autre méthode de la superclasse.

```
public class Singe {
    public void mangerBananes(int nb) { ... } ... }
class Babouin extends Singe {
    @Override
    public void mangerBananes(int nb) { ... } // redéfinition ...

}
```

- En plus de rappeler au programmeur qu'il s'agit de la redéfinition d'une autre méthode, la présence de l'annotation *@Override* va forcer le compilateur à vérifier que la méthode a été redéfinie correctement.

Affichage d'un objet

@Override

- Dans l'exemple suivant le programmeur a fait une erreur dans le nom de la méthode (oublie de la lettre "s"), erreur qui sera détectée par le compilateur qui ne trouvera pas la méthode correspondante dans la superclasse.

```
public class Singe {
    public void mangerBananes(int nb) { ... } ... }
class Babouin extends Singe {
    @Override
    public void mangerBanane(int nb) { ... } // n'est pas une redéfinition
...
}
```

04

Comparaison d’objets

04

Comparaison d'objets

Besoin

Considérons le programme Java ci-dessous:

```
public class RectangleMain
{
    public static void main(String[] args)
    {
        Rectangle r1 = new Rectangle(5.0,9.0);
        Rectangle r2 = new Rectangle(5.0,9.0);
        if (r1 == r2)
        {
            System.out.println("Rectangles identiques ");
        }
        else
        {
            System.out.println("False");
        }
    }
}
```

L'opérateur **==** appliqué à deux objets, compare les références de ces objets.

```
>>> False
```

Comparaison d'objets

equals()

- Tout comme pour l'affichage d'un objet, Java prévoit une méthode qui retourne un booléen indiquant si deux objets sont égaux ou pas.
- La méthode **equals()** est définie dans la classe Object ; en conséquence toutes les classes Java en hérite
- Pour utiliser la méthode **equals()**, il est nécessaire de la redéfinir en indiquant les attributs qui doivent être égaux pour ainsi dire que deux objets sont égaux ➔ utilisation de l'annotation @Override

Syntaxe :

```
public boolean equals(Object o)
{
        // Corps de la méthode
        return …
}
```

À l'appel →

Object X.equals(Object *autreObjet*)

Comparaison d'objets

equals()

Exemple :

```java
public class Main {
    public static void main(String[] args) {
        String myStr1 = "Hello";
        String myStr2 = "Hello";
        String myStr3 = "Another String";

System.out.println(myStr1.equals(myStr2));

System.out.println(myStr1.equals(myStr3));
    }
}
```

```
>>> true
>>> false
```

```java
public class StrMain {
    public static void main(String[] args) {
        String str1="Rouge";
        String str2="Rou";
        String str3= str2+"ge";
        if (str1.equals(str3)){
            System.out.println("chaines identiques");
        }
    }
}
```

```
>>> chaines identiques
```

Comparaison d'objets

equals()

Exemple :

```
class Rectangle {
    private double largeur;
    private double hauteur;
    // constructeur
    Rectangle(double h, double l)
    {  hauteur = h;
        largeur = l;}

    @Override
    public boolean equals(Object obj) {
        Rectangle autreRectangle = (Rectangle) obj;
        if (autreRectangle == null) {
            return false;
        }
        else {
            return (hauteur == autreRectangle.hauteur)
                    && (largeur == autreRectangle.largeur);
        }}
}
```

```
class RectangleMain{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle(5.0,9.0);
        Rectangle r2 = new Rectangle(5.0,9.0);
        if (r1.equals(r2)){
            System.out.println("2- equals = TRUE");
        }
        else{
            System.out.println("2- Not equals");
        }
    }
}
```

```
>>> 2- equals = TRUE
```

05

Attributs et méthodes statiques

05

Attributs et Méthodes Statiques

Attribut Statique

- Un attribut statique est une variable qui appartient à la classe plutôt qu'à une instance spécifique.

Syntaxe :

```
Class NomClasse
{
 public static typeAttribut nomAttributStat;
….
}
```

À l'appel →

```
NomClasse.nomAttributStat
```

Exemple :

```
public class Point {
    public static int nombreDePoints;
}
```

À l'appel →

```
Point.nombreDePoints = 10;
```

Attributs et Méthodes Statiques

Méthode Statique

- Une méthode statique est une méthode qui appartient à la classe et non à une instance.

Syntaxe :

```
Class NomClasse
{

 static typeRetourMethode nomMethode (liste param) { ….  }

}
```

À l'appel →

```
NomClasse.nomMethode(…);
```

Exemple :

```
public class Point {
    public static void afficherNombreDePoints() {
        System.out.println("Nombre de points : " +
nombreDePoints);
    }
}
```

À l'appel →

```
Point.afficherNombreDePoints();
```

Attributs et Méthodes Statiques

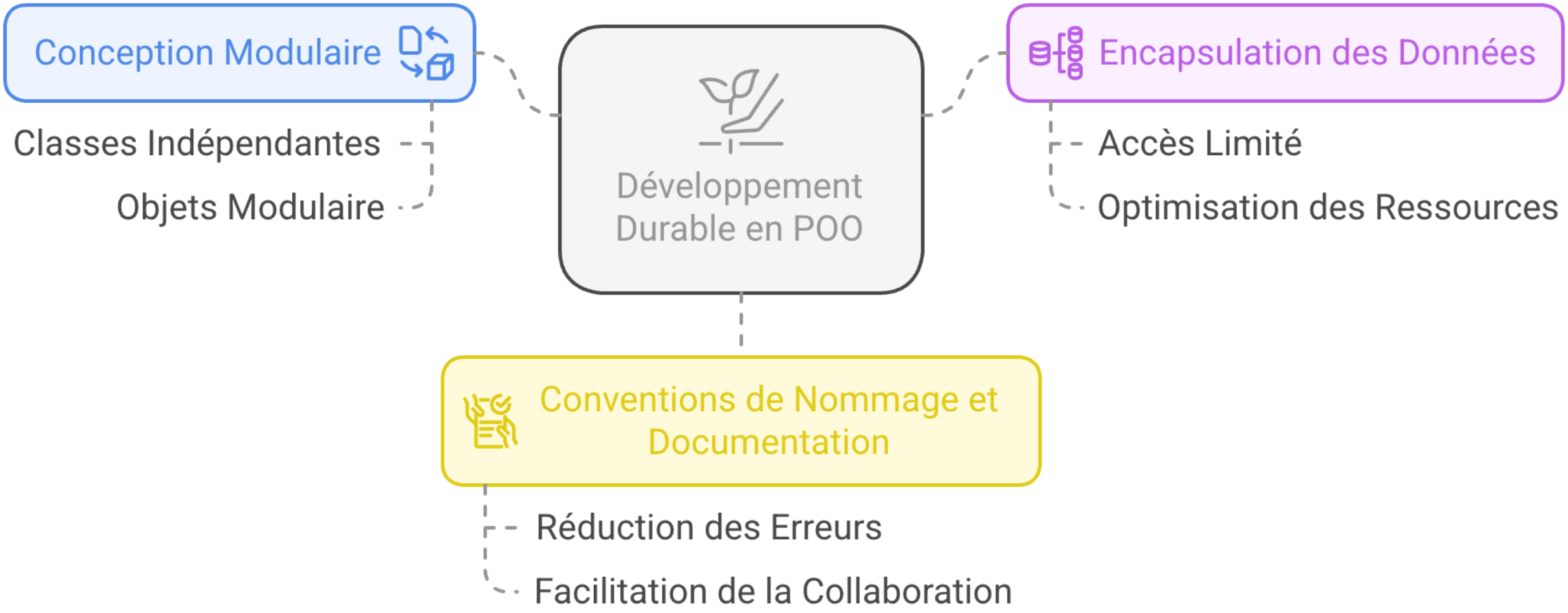
Exemple:

```java
public class Point {
    private static int nombreDePoints = 0;
    private int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
        nombreDePoints++;
    }

    public static int getNombreDePoints() {
        return nombreDePoints;
    }
}
```

```java
public class Main {
    public static void main (String [] args){
        Point p1 = new Point(1, 2);
        Point p2 = new Point(3, 4);
        System.out.println("Nombre de points
créés : " + Point.getNombreDePoints());

    }
}
```

```
>>> Nombre de points créés : 2
```

Recommandations pour des Applications Vertes en POO



Activité

1. Créez une classe **Personne** avec les attributs ***nom*** et ***age***.
2. Implémentez un ***constructeur par défaut***.
3. Implémentez un ***constructeur paramétré*** en utilisant le mot-clé **this** pour initialiser les attributs.
4. Fournissez des méthodes accesseurs et mutateurs pour ***nom*** et ***age***.
5. Redéfinissez la méthode ***toString*** pour pouvoir afficher tous les attributs de la classe **Personne**.
6. Créez une classe **TestPersonne** contenant la méthode main pour tester la fonctionnalité.
7. Chaque instance de **Personne** doit avoir un identifiant unique qui s'incrémente automatiquement à chaque nouvel objet créé. Comment pouvons-nous y parvenir ? Testez votre solution dans la classe **TestPersonne**.